ORIENTATION

# Programming for Information Systems

# Meet the course

The people and coordination problem behind our programming work

# Lecturer: Mengsay Loem

## Part-time Lecturer

2026–present

**National University of Battambang**

Cambodia

Programming for Information Systems.

## Engineer

2026–present

**Preferred Networks, Inc.**

Japan

Building large language models (LLMs).

## Researcher

2024–2026

**Sansan, Inc.**

Japan

R&D on AI for document understanding and information extraction.

## Education

**Tokyo Tech**

Japan

M.Eng. in Artificial Intelligence
B.Eng. in Computer Science

Outside work: hiking, walking, sauna and hot springs.

# A system begins with people

ILLUSTRATIVE WORKPLACE CASE

A fictional Cambodian university volunteer team helps coordinate surplus-food offers. Donors, volunteers and community partners need a clear shared record.

## Donor

What can we offer, and in what units?

## Volunteer team

What has been reserved and collected?

## Community partner

What was actually delivered?

Our case is fictional. The coordination questions are concrete.

# Course access and tools

The materials, help routes and working environment for this course

# Know what to complete—and when to stop

## Required

Complete the labelled weekly evidence and make sure you can explain it.

## Already supplied

Use the starter, programs, data and diagrams. Do not rebuild the infrastructure.

## Optional

Extra readings, examples and extensions can wait until required work is saved.

Save the required evidence and explain it; optional material can wait.

# The course package is your starting point

1

## README

Find the learning path, setup, current notebook, slides and ZIP.

2

## Live class

Use the access details announced separately for the class.

3

## Your notebook

Use Colab or a local Python environment.

4

## Reflection

Follow the form link and select the week.

Private submissions and contact: submit.progmis@gmail.com.

# Questions belong in the shared Issues space

1

## Search

Look for the same week, topic or error.

2

## Show

Give input, expected result and observed result.

3

## Discuss

Add your OS, Python version and what you tried.

4

## Close the loop

Try the suggestion and report what changed.

Ask during or after class. A clear question helps the next student too.

# Choose one working environment

## Google Colab

Use a browser and a Google account. Prepare the support files and save your own copy.

## Your computer

Use Python 3 and JupyterLab. Keep the notebook, programs, diagrams and data together.

## Prove it works

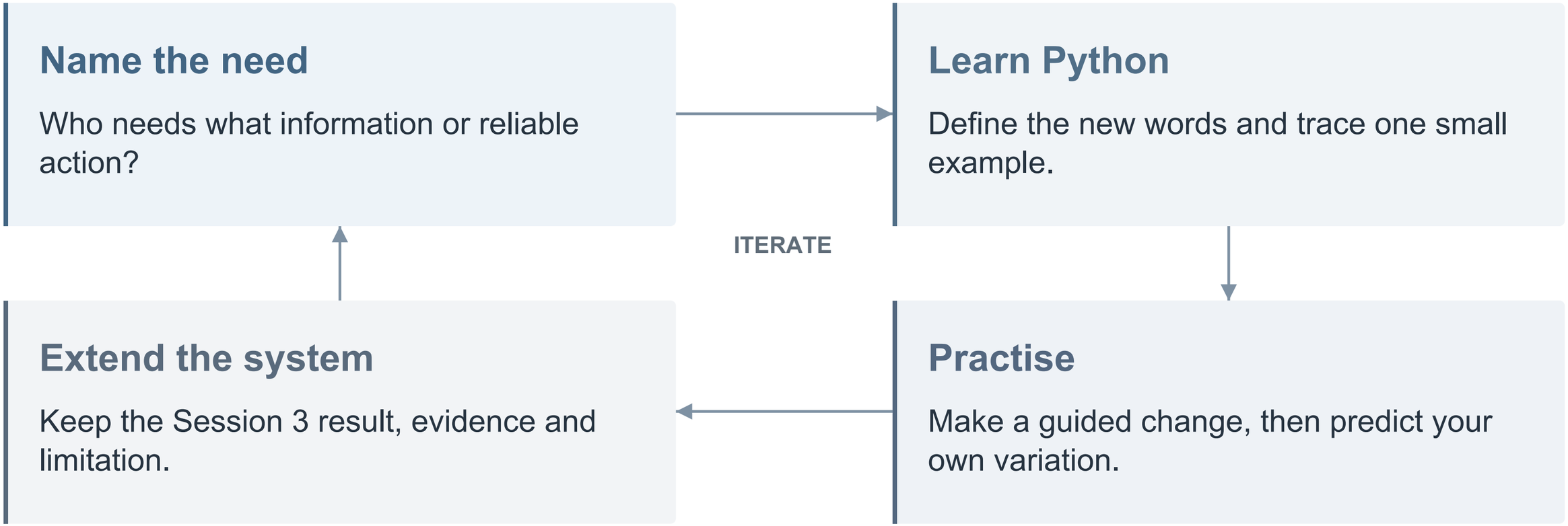
Run a cell, inspect the output, save and reopen. Report a small setup error early.

The same Python ideas apply on Windows, macOS, Linux and Colab.

# Learning and participation

How class activities, reflection and attendance evidence fit together

# A real need gives Python a purpose




Every week moves from a coordination need to a tested system checkpoint.

# Make a small-group discussion useful

## Bring an example

Show a small input, the expected result and what actually happened.

## Share the work

Rotate who runs the code, checks the result and reports the reasoning.

## Leave a trace

Write one decision, one example and one question for the class.

Explain the reasoning so another person can continue the work.

# If your connection drops

1

## Keep your work

Save your notebook or script locally.

2

## Use text

Use downloaded materials and keep a written question.

3

## Reconnect

Rejoin using the announced class access when you can.

4

## Follow up

Describe your participation honestly in the form.

A connection problem is something to explain, not a reason to invent progress.

# A short reflection closes the week

| | | |
|---|---|---|
| 1 | **Identify the week** | Open the form from Course Links; choose Week 1–10. |
| 2 | **Describe your participation** | Report live, partial, independent or no participation honestly. |
| 3 | **Show one learning step** | Name an example, a result, a question or a next step. |
| 4 | **Help improve the course** | Give brief feedback on pace, clarity and support. |

A small, specific reflection is more useful than “I understood everything.”

# Attendance evidence and feedback

## Participation record

Your name, student ID and selected week help match the response to the class roster.

## Instructor review

The instructor checks records and follows up before confirming attendance status.

## Feedback with identity

This form is identified, not anonymous. Email submit.progmis@gmail.com for personal concerns.

Submitting the form provides evidence; it does not alone prove attendance.

# Before the first programming activity

| Check | What success looks like |
| --- | --- |
| **Course package** | You can open README and the current week. |
| **Live class** | You have the current access details announced for the class. |
| **Python or Colab** | A code cell runs and shows its output. |
| **Save + reopen** | Your own file opens with your changes intact. |
| **Help route** | You can keep a written question and email submit.progmis@gmail.com. |

Tell the instructor about a setup problem early, with the error message.

# Assessment and projects

What counts, when it happens and what evidence students keep

# Proposed assessment structure

| Component | Weight | Main evidence |
| --- | --- | --- |
| **Participation and attendance** | 10% | Attendance record, participation and weekly reflection |
| **Six research assignments** | 30% | Six individual submissions worth 5% each |
| **Individual midterm** | 30% | Week 4 programming task, tests and explanation |
| **Final project** | 30% | Baseline reproduction, notes, handoff and individual evidence |

The four proposed components total 100%; university confirmation still applies.

# Six research assignments

| No. | Discussed | Focus |
|---|---|---|
| **1** | Week 2 | Goals, systems, AI-era engineer |
| **2** | Week 3 | Functions and testing |
| **3** | Week 5 | Input validation |
| **4** | Week 6 | Persistence and databases |
| **5** | Week 7 | Transactions and failure handling |
| **6** | Week 8 | Output verification, including AI |

Each assignment is worth 5%; Week 4 and Weeks 9–10 have no research submission or sharing.

# Research evidence and responsible AI use

## Investigate

Use credible sources, identify the question and check claims before relying on them.

## Discuss with AI, if used

Ask follow-up questions or challenge an answer. Keep a link, screenshots or relevant excerpts.

## Explain your reasoning

State what you understand, how the evidence supports it and what remains uncertain.

Full marks do not depend on AI, a paid tool or a presentation.

# Research sharing and marks

## Normal full marks

Submit clear evidence and your own reasoning in slides, a short report or another readable format.

## Optional volunteer sharing

Volunteers may present for a small capped bonus. The exact bonus is not yet confirmed.

## Fair opportunities

The instructor will use an equitable rotation or equivalent method before offering the bonus.

The written or visual submission can earn the normal full 5% without presenting.

# Proposed individual midterm

## Build and test

Complete one planning function and pass four common cases for totals, review precedence and the boundary.

## Package

Submit README.md, midterm.py and test_midterm.py in a ZIP another person can run.

## Explain

Run and explain your own code. State whether and where AI helped, or state that it was not used.

The proposed 30-point midterm uses Weeks 1–4 Python, a reproducible ZIP and individual explanation.

# Your final project starts before the final week

1

## Week 8 · Prepare

Identify the baseline inputs and outputs. Predict two cases; improvement ideas are optional.

2

## Week 9 · Reproduce

Reproduce, complete the notes and exchange a peer run check.

3

## Week 10 · Present

Demonstrate, hand over the system and complete individual verification.

Plain inputs → supplied function → output. Reproduce and explain; changes are optional.

# Week 10: show a system another person can use

## Explain the reproduced result

Who needs it? What is in scope? Reproduce the supplied baseline. An improvement is optional.

## Demonstrate evidence

Run acceptance cases and a boundary or failure case. Let a peer try an unseen input.

## Hand it over

Provide starting data, run instructions, expected results, recovery advice and one limitation.

Present the need, demonstrate the evidence, and explain a limitation.

# Start the course

A first conversation and optional references for the next step

# Our first conversation

ILLUSTRATIVE WORKPLACE CASE

In the chat or assigned discussion space, introduce yourself with a preferred name and one everyday coordination problem that software might help explain.

## Notice

Who needs information from whom?

## Question

Which detail is missing or ambiguous?

## Start small

What could a first working program calculate or record?

We begin with a useful question, then build a program we can explain.

# Further reading in English

1. ## Python Tutorial — introduction and first examples

   https://docs.python.org/3/tutorial/

2. ## Jupyter Notebook — running code

   https://jupyter-notebook.readthedocs.io/en/stable/examples/Notebook/Running%20Code.html

3. ## GitHub Docs — creating an issue

   https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/creating-an-issue

Read one relevant section, then bring an example and a question.